Operating Systems Assignment 5

Banker's Algorithm - Warehouse Deadlock Avoidance
(case study in C)

Name: Shaurya Dixit USN: 1DS25CG046
Subject: Operating Systems Submitted to: Prof. Nayana Shinde

This report documents a working case study of the Banker's Algorithm applied to a warehouse environment, not a bank. The assignment was to apply deadlock avoidance to a real-life application, and I chose a warehouse where forklift operators (processes) compete for shared equipment such as forklifts, pallet jacks, loading docks, and barcode scanners (resources). My program, bankers.c, is a single C file that runs in the terminal with a menu-driven interface. It loads a default warehouse scenario, displays the allocation, maximum, need, and available matrices, and runs the Banker's safety algorithm to determine whether all orders can be completed without deadlock.

The rest of this report explains what an operating system is, what deadlock means, how the Banker's Algorithm prevents deadlock, how the warehouse scenario maps onto the algorithm's matrices, the design and key snippets of the code, a step-by-step trace of the safety check using real numbers, screenshots of the program running, and likely viva questions with answers.

- Code file: bankers.c (only stdio.h and string.h, ~190 lines)
- How to build: gcc bankers.c -o bankers
- How to run: ./bankers on Linux/Mac, or bankers.exe on Windows

1. Introduction to Operating Systems

An operating system is the most important software that runs on a computer. It manages the computer's memory, processes, and all of its hardware and software. It also allows the user to communicate with the computer without knowing how to speak the computer's language. Without an operating system, a computer is useless.

The operating system acts as an intermediary between users and the computer hardware. It provides a convenient interface for users to run programs and use hardware resources. It also ensures that the computer operates efficiently and that no user program can interfere with another user program or with the operating system itself.

1.1 Major functions of an Operating System

- Process management: creating and deleting processes, scheduling them on the CPU, and handling process synchronization.
- Memory management: keeping track of which parts of memory are in use, by whom, and allocating/deallocating memory as needed.
- File management: creating, deleting, reading, writing, and organizing files on storage devices.
- Device management: handling communication between the CPU and devices such as printers, disks, and scanners via drivers.
- Security and protection: ensuring that processes cannot interfere with each other's memory or data.
- Resource allocation: deciding which process gets which resource, for how long, and in what order, to avoid deadlock.

2. Deadlock in Operating Systems

A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process. None of the processes can proceed, and the system comes to a standstill. A classic real-world analogy is two trains approaching each other on a single track: neither can move forward because the other is in the way, and neither can back up because there is no space.

2.1 The four necessary conditions for deadlock

- Mutual Exclusion: at least one resource must be held in a non-sharable mode; only one process can use it at a time.
- Hold and Wait: a process must be holding at least one resource and waiting to acquire additional resources held by other processes.
- No Preemption: resources cannot be forcibly taken away from a process; they must be released voluntarily.
- Circular Wait: there must exist a set of waiting processes such that each process is waiting for a resource held by the next process in the set.

All four conditions must hold simultaneously for deadlock to occur. If any one condition is broken, deadlock is impossible. Deadlock can be handled in three ways: prevention (make one condition impossible), avoidance (use algorithms like Banker's to ensure the system never enters an unsafe state), and detection and recovery (allow deadlock to happen, detect it, and recover by killing or rolling back processes).

3. The Banker's Algorithm

The Banker's Algorithm, designed by Edsger Dijkstra, is a deadlock avoidance algorithm used by operating systems to ensure that resource allocation never leads to deadlock. The name comes from an analogy: a banker must never lend out money in a way that leaves some customers unable to complete their transactions, because that would freeze the whole system. Similarly, an operating system must never grant resources to processes in a way that leaves some process stuck forever, waiting for resources that will never become available.

The algorithm works by checking, before granting any resource request, whether the system would still be in a safe state afterward. A safe state means there exists at least one sequence in which all processes can finish: each process gets its remaining needed resources, completes, and releases everything it was holding, which then allows the next process in the sequence to run. If no such sequence exists, the state is unsafe, and the request must be denied.

3.1 Data structures used

- Allocation matrix: how many instances of each resource are currently assigned to each process.
- Maximum matrix: the maximum number of each resource that each process may ever request during its lifetime.
- Need matrix: need = maximum - allocation; what each process still needs to finish its task.
- Available vector: how many instances of each resource are currently free and unallocated.

The safety algorithm works with a working vector called Work, initialized to Available. It scans all unfinished processes. For each process, it checks whether that process's remaining need fits inside Work (need[i][j] <= work[j] for all resource types j). If yes, the process is allowed to run to completion. When it finishes, all resources it was holding are released back into Work. The process is recorded in the Safe sequence. If a full scan finds no process that can run, the system is unsafe. If every process finishes, a safe sequence exists and the system will not deadlock.

4. Real-Life Application: Warehouse Order Picking

Imagine a warehouse that processes customer orders. Each order is handled by a forklift operator, and each operator is a process. The equipment they need - forklifts, pallet jacks, loading docks, and barcode scanners - are the resources. Not every operator needs every piece of equipment at once, and not every piece of equipment is available all the time. If two operators each hold one forklift and each needs the other's forklift to finish, neither can move and the warehouse grinds to a halt. That is deadlock.

My program models exactly this situation. There are five order-picking jobs (P0 through P4) and four types of equipment (R0 through R3). The numbers below are a concrete default scenario loaded when the program starts:

- R0: Forklifts
- R1: Pallet jacks
- R2: Loading docks
- R3: Barcode scanners

P0 (urgent electronics order) holds 0 forklifts, 1 pallet jack, 0 loading docks, 0 scanners, and may need up to 0 forklifts, 3 pallet jacks, 0 loading docks, 2 scanners. P1 (furniture delivery) holds 2 forklifts, 0 pallet jacks, 0 loading docks, 1 scanner, and may need up to 3 forklifts, 2 pallet jacks, 2 loading docks, 1 scanner. P2 (bulk appliance delivery) holds 3 forklifts, 0 pallet jacks, 1 loading dock, 0 scanners, and may need up to 3 forklifts, 0 pallet jacks, 2 loading docks, 2 scanners. P3 (grocery restock) holds 2 forklifts, 1 pallet jack, 1 loading dock, 0 scanners, and may need up to 4 forklifts, 2 pallet jacks, 2 loading docks, 0 scanners. P4 (bulk pallet wrap) holds 0 forklifts, 0 pallet jacks, 2 loading docks, 1 scanner, and may need up to 0 forklifts, 0 pallet jacks, 3 loading docks, 2 scanners. The warehouse currently has 2 forklifts, 3 pallet jacks, 1 loading dock, and 3 scanners free.

4.1 Mapping warehouse concepts to Banker's Algorithm

- Order-picking job P0..P4 = Process
- Forklifts, pallet jacks, loading docks, barcode scanners = Resource types R0..R3
- Equipment currently assigned to a job = Allocation matrix
- Maximum equipment a job might ever need = Maximum matrix
- Equipment still needed by a job to finish = Need matrix
- Equipment sitting idle in the warehouse = Available vector
- Order in which all jobs can finish without deadlock = Safe sequence
- Refusing to assign equipment if it would cause deadlock = Banker's safety check

5. Design and Important Parts of the Code

The whole project is one file, bankers.c, using only stdio.h and string.h. There are no comments in the code so that the logic stands on its own, and every function name describes what it does. The program follows the same menu-driven structure as the earlier assignments: a main loop reads a number and dispatches to the matching screen.

Snippet 1 - data structures: matrices and vectors

```
int n, m;
int alloc[MAX_P][MAX_R], max[MAX_P][MAX_R], need[MAX_P][MAX_R];
int avail[MAX_R], finish[MAX_P], safe[MAX_P];
```

What this means: n is the number of processes (order-picking jobs) and m is the number of resource types (equipment). alloc tracks what each job currently holds, max tracks what it may ever need, need is the difference (what it still needs), avail tracks what is free on the floor, finish marks which jobs have completed in the safety check, and safe records the order in which jobs finish if the system is safe.

Snippet 2 - loading the default warehouse scenario

```
void loadDefault() {
    n = 5; m = 4;
    int a[5][4] = {{0,1,0,0},{2,0,0,1},{3,0,1,0},{2,1,1,0},{0,0,2,1}};
    int mx[5][4] = {{0,3,0,2},{3,2,2,1},{3,0,2,2},{4,2,2,0},{0,0,3,2}};
    int av[4] = {2,3,1,3};
    ... copy into global tables ...
    calcNeed();
}
```

What this means: this function hard-codes a believable warehouse scenario so the demo is ready immediately. It sets 5 jobs and 4 equipment types, fills the allocation and maximum matrices, sets available resources to 2 forklifts, 3 pallet jacks, 1 loading dock, and 3 scanners, marks every job as unfinished, and calls calcNeed() to build the need matrix from max - allocation.

Snippet 3 - calculating the Need matrix

```
void calcNeed() {
    int i, j;
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
}
```

What this means: need is simply maximum minus allocation, computed for every job and every resource type. If P1's maximum for forklifts is 3 and it currently holds 2, its need for forklifts is 1. The need matrix is what the safety algorithm consults when deciding whether a job can finish with the resources currently free.

Snippet 4 - the Banker's safety algorithm

```
while (count < n) {
    found = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            for (j = 0; j < m; j++)
```

```
                if (need[i][j] > work[j]) break;
            if (j == m) {
                for (k = 0; k < m; k++) work[k] += alloc[i][k];
                safe[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if (found == 0) break;
}
```

What this means, line by line: the algorithm keeps a working copy of available resources called Work, initialized to Available. It scans all unfinished jobs. For each job, it checks whether that job's remaining need fits inside Work (need[i][j] <= work[j] for all resource types j). If yes, the job is allowed to run to completion. When it finishes, all resources it was holding are returned to Work. The job is recorded in the safe sequence. If a full scan finds no job that can run, the system is unsafe and the loop stops. If every job finishes, a safe sequence exists and the system will not deadlock.

6. Step-by-Step Trace of the Safety Check

Here is exactly what happens when the program runs option 5 on the default warehouse scenario. Available resources at the start: R0=2, R1=3, R2=1, R3=3.

Step 1: Initial check

Work = [2, 3, 1, 3]. Scan all jobs. P0 needs [0, 2, 0, 2]. Every need fits in Work (0<=2, 2<=3, 0<=1, 2<=3), so P0 can finish. P0 releases its allocation [0, 1, 0, 0]. Work becomes [2, 4, 1, 3]. P0 is added to the safe sequence.

Step 2: Second job

Work = [2, 4, 1, 3]. P1 needs [1, 2, 2, 0]. Need for R2 is 2, but Work[R2] is only 1, so P1 cannot run yet. P2 needs [0, 0, 1, 2]. Every need fits (0<=2, 0<=4, 1<=1, 2<=3), so P2 runs. P2 releases [3, 0, 1, 0]. Work becomes [5, 4, 2, 3]. P2 is added to the safe sequence.

Step 3: Third job

Work = [5, 4, 2, 3]. P1 still needs [1, 2, 2, 0], which now fits (1<=5, 2<=4, 2<=2, 0<=3), but let us continue in scan order. P3 needs [2, 1, 1, 0]. Every need fits (2<=5, 1<=4, 1<=2, 0<=3), so P3 runs. P3 releases [2, 1, 1, 0]. Work becomes [7, 5, 3, 3]. P3 is added to the safe sequence.

Step 4: Fourth job

Work = [7, 5, 3, 3]. P4 needs [0, 0, 1, 1]. Every need fits (0<=7, 0<=5, 1<=3, 1<=3), so P4 runs. P4 releases [0, 0, 2, 1]. Work becomes [7, 5, 5, 4]. P4 is added to the safe sequence.

Step 5: Fifth job

Work = [7, 5, 5, 4]. P1 needs [1, 2, 2, 0]. Every need fits (1<=7, 2<=5, 2<=5, 0<=4), so P1 runs. P1 releases [2, 0, 0, 1]. Work becomes [9, 5, 5, 5]. P1 is added to the safe sequence.

Step 6: Result

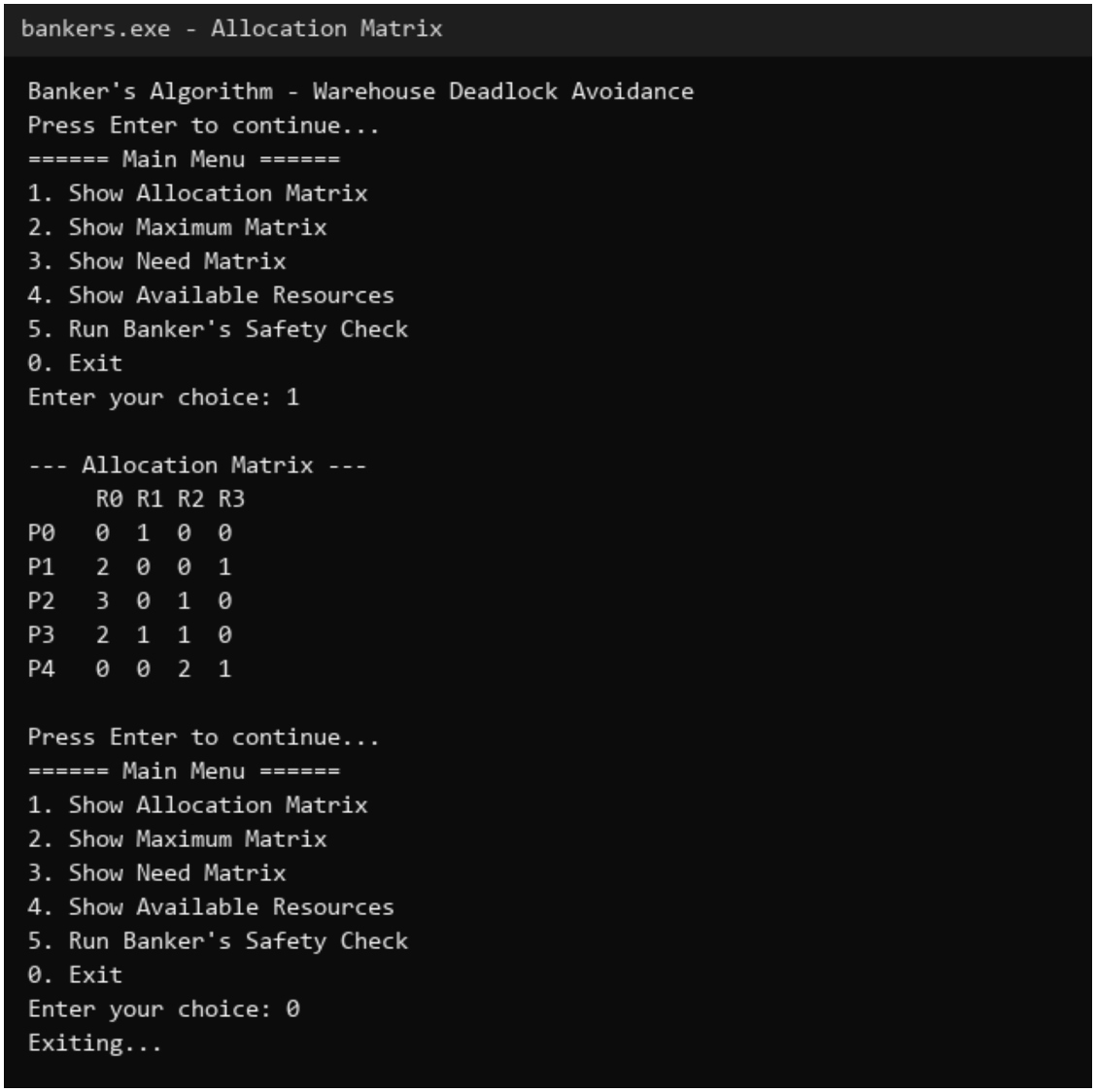
All five jobs have finished. The system is in a safe state, and the safe sequence is P0, P2, P3, P4, P1. The warehouse can complete all five orders in this order without deadlock.

7. Screenshots of the Program Running

The screenshots below were captured from real runs of bankers.exe. They show the menu, the allocation matrix, the need matrix, the available resources, and the safety check result. In a live terminal, whatever you type is echoed next to each prompt.

Output A - Allocation Matrix

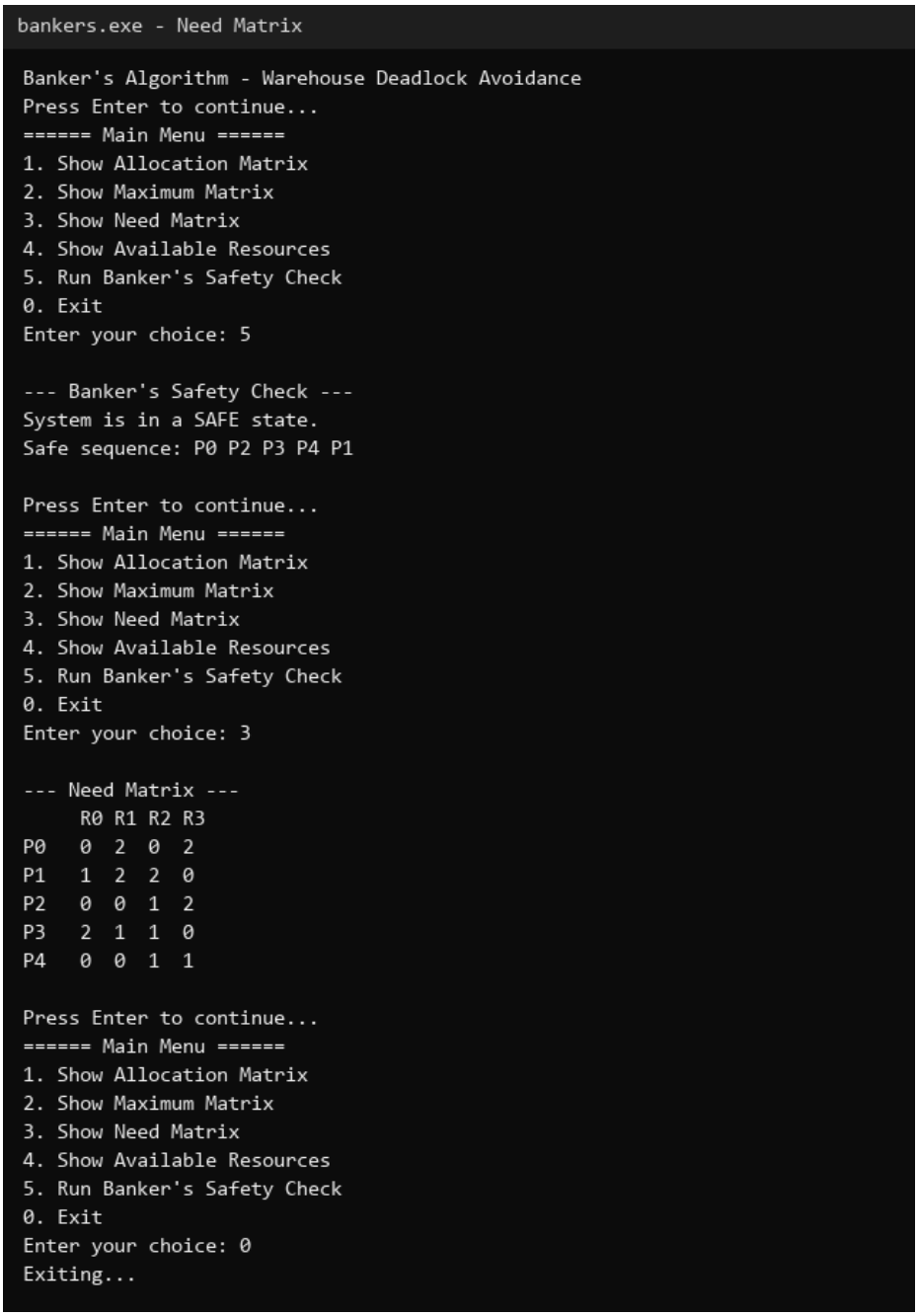
This shows what each order-picking job currently holds: P2 holds 3 forklifts and 1 loading dock, P1 holds 2 forklifts and 1 barcode scanner, and so on.



Screenshot 1 - Allocation Matrix

Output B - Need Matrix

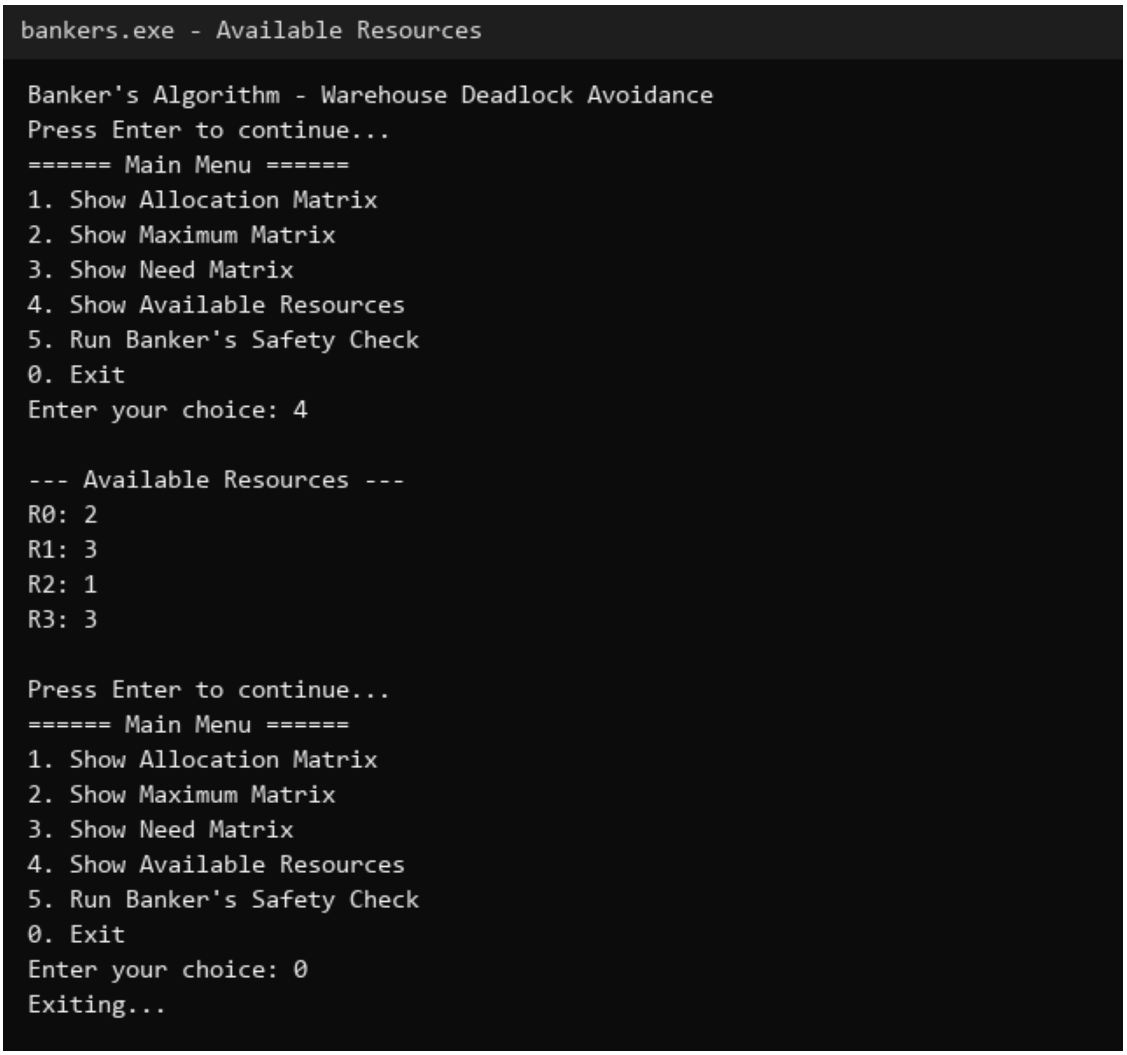
The need matrix shows what each job still needs before it can finish: P0 needs 2 pallet jacks and 2 scanners, P2 needs 1 loading dock and 2 scanners, and so on. This is computed as maximum minus allocation.



Screenshot 2 - Need Matrix

Output C - Available Resources

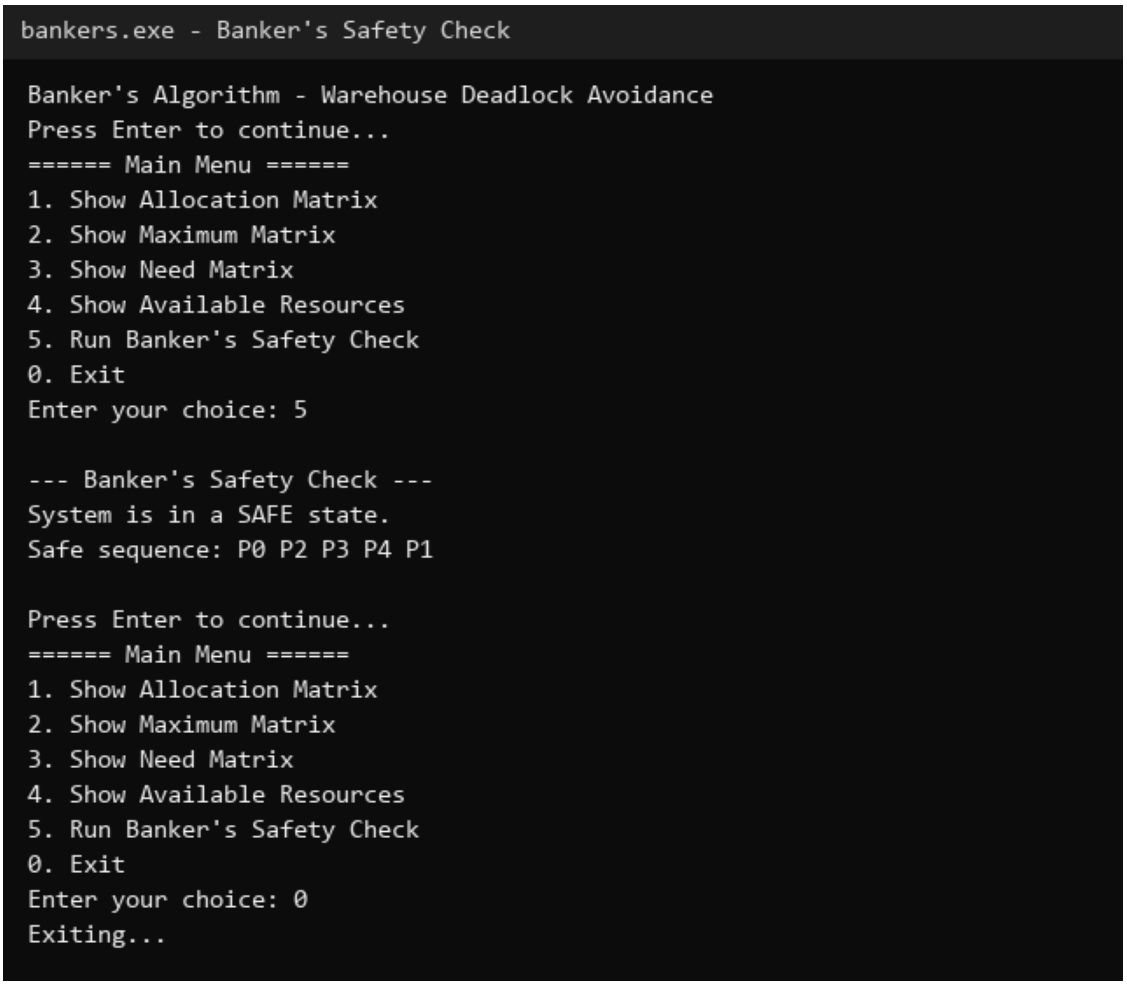
The warehouse currently has 2 forklifts, 3 pallet jacks, 1 loading dock, and 3 barcode scanners free for allocation.



Screenshot 3 - Available Resources

Output D - Banker's Safety Check

Running the safety algorithm determines that the system is in a safe state and prints the safe sequence P0 P2 P3 P4 P1. This means the warehouse can complete all five orders in this order without deadlock: P0 finishes first and releases its pallet jack, then P2 runs and releases its forklifts and loading dock, and so on until every job is done.



Screenshot 4 - Banker's Safety Check with safe sequence

8. Likely Viva Questions and Answers

- What is deadlock? A situation where a set of processes are blocked because each holds a resource and waits for another resource held by another process in the set, so none can proceed.
- What are the four conditions for deadlock? Mutual exclusion, hold and wait, no preemption, and circular wait. All four must hold at the same time.
- What is the Banker's Algorithm? A deadlock avoidance algorithm that checks whether granting a resource request keeps the system in a safe state before allocating resources.
- What is a safe state? A state in which there exists at least one sequence in which all processes can finish by getting their needed resources, completing, and releasing what they held.
- What is the difference between deadlock prevention and avoidance? Prevention makes one of the four conditions impossible; avoidance (Banker's) dynamically checks each request to ensure the system never enters an unsafe state.
- What is a safe sequence? An order of processes such that each process can get its needed resources, finish, and release them, allowing the next process to proceed.
- What data structures does your program use? Allocation, Maximum, and Need matrices (n x m), an Available vector (m), a Finish array (n), and a Safe sequence array (n).
- How is Need calculated? Need = Maximum - Allocation, for every process and every resource type.
- Why is your scenario not a bank? It is a warehouse where order-picking jobs compete for forklifts, pallet jacks, loading docks, and barcode scanners.
- What happens if the system is unsafe? The request is denied and the system does not enter deadlock.

9. How to Demo (2 minutes)

- Compile: gcc bankers.c -o bankers
- Run: ./bankers (Linux/Mac) or bankers.exe (Windows)
- Press Enter to see the boot message and main menu
- Choose 1 to show the Allocation Matrix (what each job holds)
- Choose 3 to show the Need Matrix (what each job still needs)
- Choose 4 to show Available Resources (equipment free on the floor)
- Choose 5 to run the Banker's Safety Check and reveal the safe sequence
- Explain that P0 P2 P3 P4 P1 means the warehouse completes all orders safely
- Choose 0 to exit

10. Limitations and Conclusion

Like any classroom model, this program simplifies reality. It does not handle dynamic resource requests during execution (the Banker's Algorithm can, but this version only does the initial safety check). It does not model multiple instances with complex hierarchies, preemption, or recovery from deadlock. It keeps the record-keeping side visible: creating records, searching them, filtering them, and updating states. That is exactly what this assignment asked for, and keeping it manual makes every idea visible on the screen.

In conclusion, the program meets the assignment: a working case study applying the Banker's Algorithm to a real-life application that is not a bank. It demonstrates deadlock avoidance through the allocation, maximum, need, and available matrices, computes the safe sequence, and prints whether the warehouse system is safe. The code stays minimal so that it compiles and runs anywhere, and every screen traces back to a few lines of C.

Project Link

https://github.com/SDlel/os_assignment_5